

TASK 1 – NUMBER GUESSING GAME

1. Problem Statement

Develop a Number Guessing Game in which the computer generates a random number between 1 and 100. The user repeatedly enters guesses, and the program tells the user whether the guess is correct, too high, or too low. The enhanced version limits the number of attempts, supports multiple rounds, and displays a score.

2. Objectives

- Generate a random number from 1 to 100.
- Accept the user's guess as input.
- Compare the guess with the generated number.
- Display feedback: correct, too high, or too low.
- Limit the number of attempts in each round.
- Allow the user to play multiple rounds.
- Calculate and display the score.

3. Algorithm

Step 1: Start the program.

Step 2: Set the range from 1 to 100.

Step 3: Set the maximum number of attempts.

Step 4: Generate a random number between 1 and 100.

Step 5: Ask the user to enter a guess.

Step 6: Increase the attempt counter.

Step 7: If the guess equals the secret number, display a success message and calculate the score.

Step 8: If the guess is greater than the secret number, display 'Too high'.

Step 9: If the guess is smaller than the secret number, display 'Too low'.

Step 10: Repeat until the number is guessed or the maximum attempts are used.

Step 11: Ask whether the user wants to play another round.

Step 12: If yes, start a new round; otherwise display the final score.

Step 13: Stop.

4. Java Program

```
import java.util.Random;
import java.util.Scanner;
public class NumberGuessingGame {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        int totalScore = 0;
        int round = 1;
        final int MAX_ATTEMPTS = 7;
        System.out.println("===== NUMBER GUESSING GAME =====");
        while (true) {
            int secretNumber = random.nextInt(100) + 1;
            int attempts = 0;
            boolean guessed = false;
            System.out.println("\nRound " + round);
            System.out.println("I have selected a number between 1 and 100.");
            System.out.println("You have " + MAX_ATTEMPTS + " attempts.");
            while (attempts < MAX_ATTEMPTS) {
                System.out.print("Enter your guess: ");
                if (!sc.hasNextInt()) {
                    System.out.println("Please enter a valid integer.");
                    sc.next();
                    continue;
                }
                int guess = sc.nextInt();
                attempts++;
                if (guess == secretNumber) {
                    guessed = true;
                    // More points for fewer attempts.
                    int score = (MAX_ATTEMPTS - attempts + 1) * 10;
                    totalScore += score;
                    System.out.println("Correct! You guessed the number.");
                    System.out.println("Number of attempts: " + attempts);
                    System.out.println("Round score: " + score);
                    break;
                } else if (guess > secretNumber) {
                    System.out.println("Too high! Try a smaller number.");
                } else {
                    System.out.println("Too low! Try a larger number.");
                }
                System.out.println("Attempts remaining: "
                                   + (MAX_ATTEMPTS - attempts));
            }
            if (!guessed) {
                System.out.println("\nGame over for this round.");
                System.out.println("The correct number was: " + secretNumber);
            }
            System.out.println("Total score: " + totalScore);
            System.out.print("\nDo you want to play another round? (yes/no): ");
            String choice = sc.next();
            if (!choice.equalsIgnoreCase("yes")) {
                break;
            }

            round++;
        }
        System.out.println("\n===== FINAL RESULT =====");
        System.out.println("Rounds played: " + round);
        System.out.println("Final score: " + totalScore);
        System.out.println("Thank you for playing!");
        sc.close();
    }
}
```

5. Explanation of the Program

Random number generation: `Random random = new Random();` creates a `Random` object.

`random.nextInt(100) + 1` generates an integer from 1 through 100.

Maximum attempts: `MAX_ATTEMPTS` is set to 7. Therefore, the user gets at most seven valid guesses in each round.

Input: `Scanner` is used to read the user's guess. `hasNextInt()` prevents the program from crashing when the user enters non-numeric input.

Comparison: If `guess == secretNumber`, the answer is correct. If `guess > secretNumber`, the program displays 'Too high'. Otherwise, it displays 'Too low'.

Multiple rounds: The outer while loop starts a new round when the user enters yes. Entering anything other than yes ends the game.

Score: The score is based on how quickly the user guesses the number. With seven maximum attempts, the

scores for a correct guess on attempts 1 through 7 are 70, 60, 50, 40, 30, 20, and 10 respectively.

6. Sample Output

```
===== NUMBER GUESSING GAME =====  
Round 1  
I have selected a number between 1 and 100.  
You have 7 attempts.  
Enter your guess: 50  
Too low! Try a larger number.  
Attempts remaining: 6  
Enter your guess: 75  
Too high! Try a smaller number.  
Attempts remaining: 5  
Enter your guess: 63  
Correct! You guessed the number.  
Number of attempts: 3  
Round score: 50  
Total score: 50  
Do you want to play another round? (yes/no): no  
===== FINAL RESULT =====  
Rounds played: 1  
Final score: 50  
Thank you for playing!
```

7. Features Implemented

Requirement

Implementation

Random number 1–100

Random class and `nextInt(100) + 1`

User guess

Scanner input

Feedback

Correct / Too high / Too low

Attempt limit

Maximum 7 attempts per round

Multiple rounds

Outer while loop

Score

10–70 points depending on attempts

8. Conclusion

The Number Guessing Game successfully implements all the required features. It generates a random number, accepts and evaluates guesses, provides useful feedback, limits attempts, supports multiple rounds, and keeps a running score. The program also handles invalid non-integer input safely.